FACTORYOS

Feature Feasibility Report

Prepared: July 23, 2026 · Internal Document · GDF Kenya

FactoryOS — Feature Feasibility Document

Prepared: July 2026 · Based on code review of the live codebase

Overview

This document covers four proposed features, their technical feasibility, what already exists in the codebase, what still needs to be built, and a realistic effort estimate for each.

#	Feature	Feasibility	Effort
1	Voice Input	Very Feasible	1–2 days
2	Auto-delivery from Zoom/Meet/Teams	Already ~80% Built	½ day to activate
3	Transcript → Engagement Linking	Already Built	Minor enhancement
4	Accessibility (Blind / Disability Support)	Feasible	3–5 days

Feature 1 — Voice Input on Transcript Upload

What It Does

Instead of typing or pasting a transcript, a user clicks a microphone button, speaks (or holds the device near speakers during a live call), and the text transcribes in real-time directly into the upload box. The user reviews it and submits as normal.

What Already Exists

- The transcript upload textarea is inside `EngagementDetail.jsx` — it's a standard controlled `<textarea>`. Voice input would simply write into this same field.
- The backend `/api/engagements/<id>/transcripts` endpoint in `app.py` accepts plain text — **no backend changes needed at all**.

What Needs to Be Built

Only frontend changes:

```
frontend/src/pages/EngagementDetail.jsx
  └─ Add a MicrophoneButton component next to the textarea
      ├─ Uses the Web Speech API (window.SpeechRecognition)
      ├─ On click: starts recognition, streams results into the textarea
      └─ On click again (or silence): stops recognition
```

No new dependencies. The Web Speech API is built into Chrome, Edge, and Safari — no paid service, no API key, no backend change.

Technology

```
// Browser-native — no npm package needed
const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)();
recognition.continuous = true;
recognition.interimResults = true;
recognition.lang = 'en-US'; // or 'sw-KE' for Swahili support
recognition.onresult = (e) => setTranscriptText(e.results[...]);
```

Limitations

- Captures **live speech only** — cannot retroactively transcribe a recording file
- Requires HTTPS in production (already the case via Tailscale Funnel)
- Chrome and Edge have the best support; Firefox has limited support

Effort Estimate

Task	Time
Add mic button UI to `EngagementDetail.jsx`	2–3 hours
Wire up Web Speech API with start/stop/interim display	2–3 hours
Handle browser compatibility (fallback message for Firefox)	1 hour
Total	**~1 day**

Feature 2 — Auto-delivery from Zoom / Google Meet / Teams

What It Does

When a Zoom (or Meet or Teams) meeting recorded by Fathom ends, the transcript is automatically pushed to FactoryOS — no manual download, no copy-paste. It lands in the Inbox and, if the meeting title matches a client, is attached and processed immediately.

What Already Exists — This Is Mostly Done

This system is **already fully implemented** in the codebase:

Component	File	Status
Fathom webhook receiver (`/api/webhooks/fathom`)	`app.py`	Built
HMAC signature verification	`fathom.py`	Built
Payload parsing (transcript, attendees, duration)	`fathom.py`	Built
Inbox storage (`InboundTranscript` model)	`models.py`	Built
Auto-match by client name	`app.py`	Built
Webhook registration CLI	`fathom.py`	Built
Inbox UI with Attach panel	`Inbox.jsx`	Built

What Still Needs to Be Done to Activate It

The code is built — it just needs to be **wired up to the live internet**:

Step 1 — Get a public URL for the backend

Fathom needs to POST to your server from the internet. Options: - **Tailscale Funnel** (already referenced in DEPLOY.md — recommended) - **ngrok** for quick local testing: `ngrok http 5000`

Step 2 — Add Fathom credentials to `.env`

```
FATHOM_API_KEY=<from Fathom Settings → API Access>
FATHOM_WEBHOOK_SECRET=<printed by step 3 below>
```

Step 3 — Register the webhook (one command)

```
cd backend
source venv/bin/activate
python3 fathom.py register https://your-public-url/api/webhooks/fathom
```

This prints a `whsec_...` secret — add it to `.env` as `FATHOM_WEBHOOK_SECRET`.

After this, **every Fathom-recorded Zoom/Meet/Teams call automatically pushes to the Inbox.**

Platform Support

Platform	Support
Zoom	Fathom records natively
Google Meet	Fathom records natively
Microsoft Teams	Fathom records natively
In-person meetings	Fathom can't join — use Voice Input (Feature 1)

Effort Estimate

Task	Time
Set up public URL (Tailscale Funnel)	30 min
Add API key + register webhook	15 min
Test end-to-end with a real meeting	30 min
Total	**~1.5 hours**

Feature 3 — How Transcripts Are Linked to the Right Engagement

The Problem

When a transcript arrives automatically from Fathom, the system needs to know: *which engagement does this belong to?*

What Already Exists

The Inbox already has **two mechanisms** built in:

Mechanism A — Automatic Name Matching (already live)

The webhook handler checks if the meeting title exactly matches a client name. If it does, it auto-attaches and starts processing immediately. The transcript card in the Inbox shows `auto-matched` in green.

Mechanism B — Manual Attach Panel (already live)

If auto-match fails (meeting title doesn't match), the transcript sits in the Inbox as `pending`. A Producer or Account Manager clicks **"Attach"** on that card, picks either: - An **existing engagement** (dropdown of all engagements in an attachable state), or - Creates a **new engagement** for a specific client

The system then processes the transcript immediately.

What Could Be Enhanced

The current auto-match requires an **exact** client name in the meeting title, which is fragile. Two improvements worth adding:

Enhancement A — Fuzzy Name Matching

```
# backend/app.py — webhook handler
from difflib import SequenceMatcher

def _fuzzy_match_client(title):
    clients = db_session.query(Client).all()
    for client in clients:
        ratio = SequenceMatcher(None, title.lower(), client.name.lower()).ratio()
        if ratio > 0.75:   # 75% similarity threshold
            return client
    return None
```

This would match "Safaricom Q3 Meeting" to the "Safaricom" client automatically.

Enhancement B — Engagement Code in Meeting Title

The producer adds an engagement code to the Zoom/calendar invite title, e.g.:

> `[ENG-0042] Safaricom Q3 Review`

The webhook reads the `[ENG-XXXX]` tag and attaches directly to that engagement — zero ambiguity. This is the most reliable approach for large teams.

Effort Estimate

Task	Time
Fuzzy name matching in webhook handler	2–3 hours
Engagement code tag parsing	1–2 hours
Total	**~half a day**

Feature 4 — Accessibility (Blind / Visual Impairment / Disability Support)

What It Means

Making FactoryOS usable for people who: - Use a **screen reader** (NVDA, VoiceOver, JAWS) — software that reads the screen aloud - Navigate by **keyboard only** (Tab, Enter, Arrow keys) — no mouse - Have **low vision** and need high contrast or larger text - Use **voice control** software (e.g. Dragon NaturallySpeaking)

Current State

The app has minimal accessibility support right now. Key gaps: - Most buttons lack `aria-label` (screen readers will just say "button") - Dynamic content (new brief generated, error messages) doesn't announce to screen readers - Focus indicators may be invisible (common with custom CSS) - The PDF download links have no meaningful description

What Needs to Be Built

Priority 1 — Screen Reader Support (High Impact)

Add `aria-label`, `role`, and `aria-live` attributes to the key interactive areas.

```
// Example fix in EngagementDetail.jsx
<button
  aria-label="Upload transcript to generate production brief"
  onClick={handleUpload}
>
  <Upload size={14} /> Upload
</button>

// When brief is generated, announce it to screen readers:
<div aria-live="polite" aria-atomic="true">
  {isProcessing ? 'Generating brief, please wait…' : ''}
  {brief ? 'Production brief is ready.' : ''}
</div>
```

Priority 2 — Keyboard Navigation

Every interactive element must be reachable by Tab and activated by Enter/Space: - Modals and slide-out panels must trap focus while open and return it when closed - The `Escape` key must close any open modal/panel

Priority 3 — High Contrast & Text Scaling

```
/* index.css additions */
@media (prefers-contrast: high) {
  :root {
    --color-text-primary: #ffffff;
    --color-border: #ffffff;
    --color-accent: #ffcc00;
  }
}

@media (prefers-reduced-motion: reduce) {
  * { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; }
}
```

Priority 4 — Voice Input (Feature 1) also helps blind users

A blind user can speak the transcript instead of typing it — Feature 1 serves a dual purpose here.

Screen-Reader Friendly Page Structure

Pages need proper heading hierarchy (one `<h1>` per page, logical `<h2>/<h3>` nesting) and semantic HTML (`<main>`, `<nav>`, `<section>`, `<article>`).

Effort Estimate

Task	Time
Add `aria-label` to all buttons and interactive controls	1 day
Add `aria-live` regions for dynamic content (processing, errors)	half day
Fix keyboard navigation + focus trapping in modals	1 day
Add `prefers-contrast` and `prefers-reduced-motion` CSS	half day
Full accessibility audit + fix remaining issues	1 day
Total	**3–4 days**

Recommended Build Order

Given effort and impact:

```
Week 1
  Day 1:  Activate Fathom webhook (Feature 2) — biggest return for least work
  Day 2:  Add fuzzy matching + engagement code linking (Feature 3 enhancement)
  Day 3:  Voice input mic button (Feature 1)

Week 2
  Day 4–7: Accessibility pass (Feature 4) — screen readers, keyboard nav, contrast
```

Summary Table

Feature	Backend Changes	Frontend Changes	New Dependencies	Effort
Voice Input	None	`EngagementDetail.jsx` only	None	~1 day
Fathom Auto-delivery	Config only (webhook registration)	None	None	~1.5 hrs
Transcript Linking	Small enhancement to webhook handler	None	None	~half day
Accessibility	None	All pages (aria attrs, CSS)	None	3–4 days

> [!IMPORTANT] None of these features require changes to the AI agents, the database schema, or any paid third-party service beyond what is already in use. All four can be built entirely within the existing stack.
